CS-462 – Programming Language Foundations
Lecture #10: The λ-Calculus

EPFL

Lambda calculus (or λ-calculus) was introduced by Alonzo Church in the 1930s, and developed with his students Stephen Kleene and J. Barkley Rosser, to describe functions in an unambiguous and compact manner. Many real languages are based on the lambda calculus, including Lisp, Scheme, Haskell, and ML. A key characteristic of these languages is that functions are values, just like integers and booleans are values: functions can be used as arguments to functions, and can be returned from functions.

The name "lambda calculus" comes from the use of the Greek letter lambda (λ) in function definitions. (The letter lambda has no significance.) "Calculus" means a method of calculating by the symbolic manipulation of expressions.

Intuitively, a function is a rule for determining a value from an argument. Some examples of functions in mathematics are

$$f(x) = x^3$$
$$g(y) = y^3 - 2y^2 + 5y - 6.$$

1 Syntax

The pure λ-calculus contains just function definitions (called *abstractions*), variables, and function *application* (i.e., applying a function to an argument). If we add additional data types and operations (such as integers and addition), we have an *applied* λ-calculus. In the following text, we will sometimes assume that we have integers and addition in order to give more intuitive examples.

The syntax of the pure λ-calculus is defined as follows.

$e ::= x$	variable
$\mid \lambda x.\, e$	abstraction
$\mid e_1\ e_2$	application

An abstraction $\lambda x.\, e$ is a function: variable x is the *argument*, and expression e is the *body* of the function. Note that the function $\lambda x.\, e$ doesn't have a name. Assuming we have integers and arithmetic operations, the expression $\lambda x.\, x^2$ is a function that takes an argument x and returns the square of x.

An application $e_1\ e_2$ requires that e_1 is (or evaluates to) a function, and then applies the function to the expression e_2. For example, $(\lambda x.\, x^2)\ 5$ is, intuitively, equal to 25, the result of applying the squaring function $\lambda x.\, x^2$ to 5.

Here are some examples of lambda calculus expressions.

$\lambda x.\, x$	a lambda abstraction called the *identity function*
$\lambda x.\, f\ (g\ x)$	another abstraction
$(\lambda x.\, x)\ 42$	an application
$\lambda y.\, \lambda x.\, x$	an abstraction that ignores its argument and returns the identity function

Lambda expressions extend as far to the right as possible. For example $\lambda x.\, x\ \lambda y.\, y$ is the same as $\lambda x.\, x\ (\lambda y.\, y)$, and is not the same as $(\lambda x.\, x)\ (\lambda y.\, y)$. Application is left associative. For example $e_1\ e_2\ e_3$ is the same as $(e_1\ e_2)\ e_3$. In general, use parentheses to make the parsing of a lambda expression clear if you are in doubt.

1.1 Variable binding and α-equivalence

An occurrence of a variable in an expression is either *bound* or *free*. An occurrence of a variable x in a term is bound if there is an enclosing $\lambda x.\, e$; otherwise, it is *free*. A *closed term* is one in which all identifiers are bound.

Consider the following term:

$$\lambda x.\, (x\ (\lambda y.\, y\ a)\ x)\ y$$

Both occurrences of x are bound, the first occurrence of y is bound, the a is free, and the last y is also free, since it is outside the scope of the λy.

If a program has some variables that are free, then you do not have a complete program as you do not know what to do with the free variables. Hence, a well formed program in lambda calculus is a closed term.

The symbol λ is a *binding operator*, as it binds a variable within some scope (i.e., some part of the expression): variable x is bound in e in the expression $\lambda x.\, e$.

The name of bound variables is not important. Consider the mathematical integrals $\int_0^7 x^2 dx$ and $\int_0^7 y^2 dy$. They describe the same integral, even though one uses variable x and the other uses variable y in their definition. The meaning of these integrals is the same: the bound variable is just a placeholder. In the same way, we can change the name of bound variables without changing the meaning of functions. Thus $\lambda x.\, x$ is the same function as $\lambda y.\, y$. Expressions e_1 and e_2 that differ only in the name of bound variables are called *α-equivalent*, sometimes written $e_1 \equiv_\alpha e_2$.

1.2 Higher-order functions

In lambda calculus, functions are values: functions can take functions as arguments and return functions as results. In the pure lambda calculus, every value is a function, and every closed program that terminates produces a function!

For example, the following function takes a function f as an argument, and applies it to the value 42.

$$\lambda f.\, f\ 42$$

This function takes an argument v and returns a function that applies its own argument (a function) to v.

$$\lambda v.\, \lambda f.\, (f\ v)$$

2 Semantics

2.1 β-equivalence

Application $(\lambda x.\, e_1)\ e_2$ applies the function $\lambda x.\, e_1$ to e_2. In some ways, we would like to regard the expression $(\lambda x.\, e_1)\ e_2$ as equivalent to the expression e_1 where every (free) occurrence of x is replaced with e_2. For example, we would like to regard $(\lambda x.\, x^2)\ 5$ as equivalent to 5^2.

We write $e_1\{e_2/x\}$ to mean expression e_1 with all free occurrences of x replaced with e_2. There are several different notations to express this substitution, including $[x \mapsto e_2]e_1$ (used by Pierce), $[e_2/x]e_1$ (used by Mitchell), and $e_1[e_2/x]$ (used by Winskel).

Using our notation, we would like expressions $(\lambda x.\, e_1)\ e_2$ and $e_1\{e_2/x\}$ to be equivalent.

This equivalence, between $(\lambda x.\, e_1)\ e_2$ and $e_1\{e_2/x\}$, is called *β-equivalence*. Rewriting $(\lambda x.\, e_1)\ e_2$ into $e_1\{e_2/x\}$ is called a *β-reduction*. Given a lambda calculus expression, we may, in general, be able to perform β-reductions. This corresponds to executing a lambda calculus expression.

There may be more than one possible way to β-reduce an expression. Consider, for example, $(\lambda x.\, x + x)\ ((\lambda y.\, y)\ 5)$. We could use β-reduction to get either $((\lambda y.\, y)\ 5) + ((\lambda y.\, y)\ 5)$ or $(\lambda x.\, x + x)\ 5$. The order in which we perform β-reductions results in different semantics for the lambda calculus.

2.2 Call-by-value

Call-by-value (or CBV) semantics makes sure that functions are only called on values. That is, given an application $(\lambda x.\, e_1)\ e_2$, CBV semantics makes sure that e_2 is a value before calling the function.

So, what is a value? In the pure lambda calculus, any abstraction is a value. Remember, an abstraction $\lambda x.\, e$ is a function; in the pure lambda calculus, the only values are functions. In an applied lambda calculus with integers and arithmetic operations, values also include integers. Intuitively, a value is an expression that can not be reduced/executed/simplified any further.

We can give small step operational semantics for call-by-value execution of the lambda calculus. Here, v can be instantiated with any value (e.g., a function).

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{e \to e'}{v\ e \to v\ e'} \qquad \beta\text{-REDUCTION}\ \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}$$

We can see from these rules that, given an application $e_1\ e_2$, we first evaluate e_1 until it is a value, then we evaluate e_2 until it is a value, and then we apply the function to the value—a β-reduction.

Let's consider some examples. (These examples use an applied lambda calculus that also includes reduction rules for arithmetic expressions.)

$$\begin{aligned}(\lambda x.\, \lambda y.\, y\ x)\ (5 + 2)\ \lambda x.\, x + 1 &\to (\lambda x.\, \lambda y.\, y\ x)\ 7\ \lambda x.\, x + 1\\ &\to (\lambda y.\, y\ 7)\ \lambda x.\, x + 1\\ &\to (\lambda x.\, x + 1)\ 7\\ &\to 7 + 1\\ &\to 8\end{aligned}$$

$$\begin{aligned}(\lambda f.\, f\ 7)\ ((\lambda x.\, x\ x)\ \lambda y.\, y) &\to (\lambda f.\, f\ 7)\ ((\lambda y.\, y)\ (\lambda y.\, y))\\ &\to (\lambda f.\, f\ 7)\ (\lambda y.\, y)\\ &\to (\lambda y.\, y)\ 7\\ &\to 7\end{aligned}$$

2.3 Call-by-name

Call-by-name (or CBN) semantics applies the function as soon as possible. The small step operational semantics are a little simpler, as they do not need to ensure that the expression to which a function is applied is a value.

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \beta\text{-REDUCTION}\ \frac{}{(\lambda x.\, e_1)\ e_2 \to e_1\{e_2/x\}}$$

Let's consider the same examples we used for CBV.

$$\begin{aligned}(\lambda x.\, \lambda y.\, y\ x)\ (5+2)\ \lambda x.\, x+1 &\to (\lambda y.\, y\ (5+2))\ \lambda x.\, x+1\\ &\to (\lambda x.\, x+1)\ (5+2)\\ &\to (5+2)+1\\ &\to 7+1\\ &\to 8\end{aligned}$$

$$\begin{aligned}(\lambda f.\, f\ 7)\ ((\lambda x.\, x\ x)\ \lambda y.\, y) &\to ((\lambda x.\, x\ x)\ \lambda y.\, y)\ 7\\ &\to ((\lambda y.\, y)\ (\lambda y.\, y))\ 7\\ &\to (\lambda y.\, y)\ 7\\ &\to 7\end{aligned}$$

Note that the answers are the same, but the order of evaluation is different. (Later we will see languages where the order of evaluation is important, and may result in different answers.)

3 Lambda calculus evaluation

There are many different evaluation strategies for the λ-calculus. The most permissive is *full β reduction*, which allows any *redex*—i.e., any expression of the form $(\lambda x.\, e_1)\ e_2$—to step to $e_1\{e_2/x\}$ at any time. It is defined formally by the following small-step operational semantics rules:

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{e_2 \to e_2'}{e_1\ e_2 \to e_1\ e_2'} \qquad \frac{e_1 \to e_1'}{\lambda x.\, e_1 \to \lambda x.\, e_1'} \qquad \beta\ \frac{}{(\lambda x.\, e_1)\ e_2 \to e_1\{e_2/x\}}$$

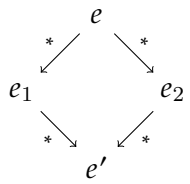
The call-by-value and call-by-name strategies from the previous section are more restrictive. Neither allows evaluation under a λ. CBV only allows an application to reduce after its argument has been reduced to a value (i.e., a λ-abstraction), while CBN allows an application to reduce even when its argument is not a value. Unlike full β reduction, both are deterministic: every expression can take at most one step.

4 Confluence

It is not hard to see that the full β reduction strategy is non-deterministic. This raises an interesting question: do the choices made during the evaluation of an expression affect the final result? The answer turns out to be no: full β reduction is *confluent* in the following sense:

Theorem (Church–Rosser)**.** *If $e \to^* e_1$ and $e \to^* e_2$ then there exists e' such that $e_1 \to^* e'$ and $e_2 \to^* e'$.*

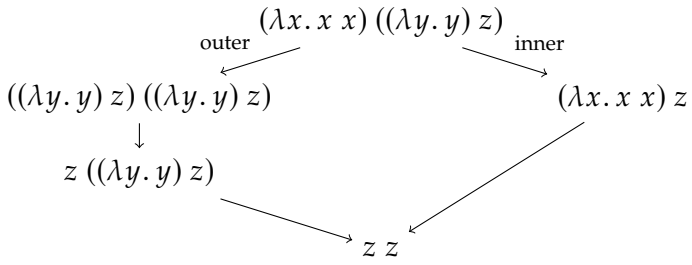
Confluence can be depicted graphically as follows:



This theorem is due to Church and Rosser [2], and confluence is often also called the Church–Rosser property. An important consequence is that every expression has at most one *normal form*—an expression that cannot step—up to α-equivalence. If $e \to^* e_1$ and $e \to^* e_2$ where e_1 and e_2 are both normal forms, then confluence gives an e' that both reduce to, which must be e_1 and e_2 themselves.

Confluence does *not* say that every reduction sequence reaches the normal form. For example, $(\lambda x.\, \lambda y.\, y)\ \Omega$, where Ω is the diverging expression discussed in Section 7, can step to the normal form $\lambda y.\, y$, but it can also step to itself forever by reducing Ω.

Interestingly, the confluence property does not hold for single steps. For example, consider the expression $e = (\lambda x.\, x\ x)\ ((\lambda y.\, y)\ z)$, which has two redexes:



Reducing the outer redex duplicates the inner one, so the left path needs two more steps to reach $z\ z$ while the right path needs only one. Hence there is no way to complete the diamond with single steps on both sides, and a naive proof of confluence by induction on single steps does not work. The standard proof, due to Tait and Martin-Löf [1], instead uses *parallel reduction*, which contracts any set of redexes in an expression at once. Parallel reduction does have the diamond property, and its reflexive-transitive closure coincides with $\to^*$, so confluence follows. Takahashi [3] gives a particularly short version of this proof.

5 Substitution

Each of the evaluation relations for λ-calculus has a β rule defined in terms of a substitution operation on expressions. Because the expressions involved in the substitution may share some variable

names (and because we are working up to α-equivalence) the definition of this operation is slightly subtle and defining it precisely turns out to be trickier than might first appear.

As a first attempt, consider an obvious (but incorrect) definition of the substitution operator. Here we are substituting e for x in some other expression:

$$\begin{array}{rcll} y\{e/x\} & = & \begin{cases} e & \text{if } y = x \\ y & \text{otherwise} \end{cases} & \\ (e_1\ e_2)\{e/x\} & = & (e_1\{e/x\})\ (e_2\{e/x\}) & \\ (\lambda y.e_1)\{e/x\} & = & \lambda y.e_1\{e/x\} & \text{where } y \neq x \end{array}$$

The intuitive idea is that the last rule relies on α-equivalence to "rewrite" abstractions that use x so they do not conflict. Unfortunately, this definition produces the wrong results when we substitute an expression with free variables under a λ. For example,

$$(\lambda y.x)\{y/x\} = (\lambda y.y)$$

To fix this problem, we need to revise our definition so that when we substitute under a λ we do not accidentally bind variables in the expression we are substituting. The following definition correctly implements *capture-avoiding substitution*:

$$\begin{array}{rcll} y\{e/x\} & = & \begin{cases} e & \text{if } y = x \\ y & \text{otherwise} \end{cases} & \\ (e_1\ e_2)\{e/x\} & = & (e_1\{e/x\})\ (e_2\{e/x\}) & \\ (\lambda y.e_1)\{e/x\} & = & \lambda y.(e_1\{e/x\}) & \text{where } y \neq x \text{ and } y \notin \mathit{fv}(e) \end{array}$$

Note that in the case for λ-abstractions, we require that the bound variable y be different from the variable x we are substituting for and that y not appear in the free variables of e, the expression we are substituting. Because we work up to α-equivalence, we can always pick y to satisfy these side conditions. For example, to calculate $(\lambda z.x\ z)\{(w\ y\ z)/x\}$ we first rewrite $\lambda z.x\ z$ to $\lambda u.x\ u$ and then apply the substitution, obtaining $\lambda u.(w\ y\ z)\ u$ as the result.

6 λ-calculus encodings

The pure λ-calculus contains only functions as values. It is not exactly easy to write large or interesting programs in the pure λ-calculus. We can however encode objects, such as booleans, and integers.

6.1 Booleans

Let us start by encoding constants and operators for booleans. That is, we want to define functions TRUE, FALSE, AND, NOT, IF, and other operators that behave as expected. For example:

$$\begin{aligned} \mathsf{AND\ TRUE\ FALSE} &= \mathsf{FALSE} \\ \mathsf{NOT\ FALSE} &= \mathsf{TRUE} \\ \mathsf{IF\ TRUE}\ e_1\ e_2 &= e_1 \\ \mathsf{IF\ FALSE}\ e_1\ e_2 &= e_2 \end{aligned}$$

Let's start by defining TRUE and FALSE:

$$\begin{aligned}\mathsf{TRUE} &\triangleq \lambda x.\, \lambda y.\, x \\ \mathsf{FALSE} &\triangleq \lambda x.\, \lambda y.\, y\end{aligned}$$

Thus, both TRUE and FALSE are functions that take two arguments; TRUE returns the first, and FALSE returns the second. We want the function IF to behave like

$$\lambda b.\, \lambda t.\, \lambda f.\, \text{if } b = \mathsf{TRUE} \text{ then } t \text{ else } f.$$

The definitions for TRUE and FALSE make this very easy.

$$\mathsf{IF} \triangleq \lambda b.\, \lambda t.\, \lambda f.\, b\ t\ f$$

Definitions of other operators are also straightforward.

$$\begin{aligned}\mathsf{NOT} &\triangleq \lambda b.\, b\ \mathsf{FALSE}\ \mathsf{TRUE} \\ \mathsf{AND} &\triangleq \lambda b_1.\, \lambda b_2.\, b_1\ b_2\ \mathsf{FALSE} \\ \mathsf{OR} &\triangleq \lambda b_1.\, \lambda b_2.\, b_1\ \mathsf{TRUE}\ b_2\end{aligned}$$

6.2 Church numerals

Church numerals encode a number n as a function that takes f and x, and applies f to x n times.

$$\begin{aligned}\overline{0} &\triangleq \lambda f.\, \lambda x.\, x \\ \overline{1} &\triangleq \lambda f.\, \lambda x.\, f\ x \\ \overline{2} &\triangleq \lambda f.\, \lambda x.\, f\ (f\ x) \\ \mathsf{SUCC} &\triangleq \lambda n.\, \lambda f.\, \lambda x.\, f\ (n\ f\ x)\end{aligned}$$

In the definition for SUCC, the expression $n\ f\ x$ applies f to x n times (assuming that variable n is the Church encoding of the natural number n). We then apply f to the result, meaning that we apply f to x $n+1$ times.

Given the definition of SUCC, we can easily define addition. Intuitively, the natural number $n_1 + n_2$ is the result of applying the successor function n_1 times to n_2.

$$\mathsf{PLUS} \triangleq \lambda n_1.\, \lambda n_2.\, n_1\ \mathsf{SUCC}\ n_2$$

Similarly, multiplication adds n_2 to $\overline{0}$, n_1 times, and a test for zero applies the constant function returning FALSE to TRUE n times:

$$\begin{aligned}\mathsf{TIMES} &\triangleq \lambda n_1.\, \lambda n_2.\, n_1\ (\mathsf{PLUS}\ n_2)\ \overline{0} \\ \mathsf{ISZERO} &\triangleq \lambda n.\, n\ (\lambda z.\, \mathsf{FALSE})\ \mathsf{TRUE}\end{aligned}$$

A predecessor function PRED, with $\mathsf{PRED}\ \overline{n+1} = \overline{n}$ and $\mathsf{PRED}\ \overline{0} = \overline{0}$, is also definable, but it is surprisingly tricky.

7 Nontermination

Consider the expression $(\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$, which we will refer to as Ω for brevity. Let's try evaluating Ω:

$$\begin{array}{rcl} \Omega & = & (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \\ & \to & (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \\ & = & \Omega \end{array}$$

Evaluating Ω never reaches a value! It is an infinite loop!

What happens if we use Ω as an actual argument to a function? Consider this program:

$$(\lambda x.(\lambda y.y))\ \Omega$$

If we use CBV semantics to evaluate the program, we must reduce Ω to a value before we can apply the function. But Ω never evaluates to a value, so we can never apply the function. Thus, under CBV semantics, this program does not terminate. If we use CBN semantics, we can apply the function immediately, without needing to reduce the actual argument to a value:

$$(\lambda x.(\lambda y.y))\ \Omega \to_{\text{CBN}} \lambda y.y$$

This difference highlights the fact that evaluation orders, like CBV and CBN, can differ in terms of termination: expressions that diverge under one evaluation order may terminate in another. This does not contradict confluence: the CBV evaluation of this program simply never finishes, so it never reaches a different result.

8 Recursion

Ω demonstrates that we can write nonterminating λ-terms, but so far they are not very useful. Can we write *recursive* functions in the λ-calculus that do actually terminate and produce a value, as we can in most "real" programming languages? It seems difficult at first because all functions are anonymous—there is no name that a function can use to refer to itself. Surprisingly, however, it is possible to make functions that behave recursively. Roughly, our strategy will build on the "self-replicating" behavior of Ω to let functions invoke copies of themselves an unbounded number of times.

Let's consider how we would like to write a factorial function:

$$\mathsf{FACT} \triangleq \lambda n.\, \mathsf{IF}\ (\mathsf{ISZERO}\ n)\ \overline{1}\ (\mathsf{TIMES}\ n\ (\mathsf{FACT}\ (\mathsf{PRED}\ n)))$$

In slightly more readable notation, this is just:

$$\mathsf{FACT} \triangleq \lambda n.\, \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times \mathsf{FACT}\ (n-1)$$

Here, as in the definition above, the name FACT is simply meant to be shorthand for the expression on the right-hand side of the equation. But FACT appears on the right-hand side of the equation as well! Trying to expand the definition would yield an infinite term. This is not a definition, it's a recursive equation.

8.1 Recursion Removal Trick

We can perform a "trick" to define a function FACT that satisfies the recursive equation above. First, let's define a new function FACT′ that looks like FACT, but takes an additional argument f. We assume that the function f will be instantiated with FACT′ itself.

$$\mathsf{FACT}' \triangleq \lambda f.\, \lambda n.\, \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (f\ f\ (n-1))$$

Note that when we call f, we pass it a copy of itself, preserving the assumption that the actual argument for f will be FACT′. Now we can define the factorial function FACT in terms of FACT′.

$$\mathsf{FACT} \triangleq \mathsf{FACT}'\ \mathsf{FACT}'$$

Let's try evaluating FACT on an integer.

$$\begin{aligned}
\mathsf{FACT}\ 3 &= (\mathsf{FACT}'\ \mathsf{FACT}')\ 3 && \text{Definition of FACT}\\
&= ((\lambda f.\, \lambda n.\, \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (f\ f\ (n-1)))\ \mathsf{FACT}')\ 3 && \text{Definition of FACT}'\\
&\to (\lambda n.\, \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (\mathsf{FACT}'\ \mathsf{FACT}'\ (n-1)))\ 3 && \text{Application to FACT}'\\
&\to \textbf{if } 3 = 0 \textbf{ then } 1 \textbf{ else } 3 \times (\mathsf{FACT}'\ \mathsf{FACT}'\ (3-1)) && \text{Application to } n\\
&\to 3 \times (\mathsf{FACT}'\ \mathsf{FACT}'\ (3-1)) && \text{Evaluating } \textbf{if}\\
&\to^* \ldots\\
&\to^* 3 \times 2 \times 1 \times 1\\
&\to^* 6
\end{aligned}$$

So we now have a technique for writing a recursive function f: write a function f' that explicitly takes a copy of itself as an argument, and then define $f \triangleq f'\ f'$.

8.2 Fixed point combinators

There is another way of writing recursive functions: we can express the recursive function as the fixed point of some other, higher-order function, and then take its fixed point. We saw this technique earlier in the course when we defined the denotational semantics for **while** loops.

Let's consider the factorial function again. The factorial function FACT is a fixed point of the following function.

$$G \triangleq \lambda f.\, \lambda n.\, \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (f\ (n-1))$$

Recall that if g is a fixed point of G, then we have $G\ g = g$. So if we had some way of finding a fixed point of G, we would have a way of defining the factorial function FACT.

There are a number of "fixed point combinators," and the (infamous) Y combinator is one of them. Thus, we can define the factorial function FACT to be simply $\mathsf{Y}\ G$, the fixed point of G. The Y combinator is defined as

$$\mathsf{Y} \triangleq \lambda f.\, (\lambda x.\, f\ (x\ x))\ (\lambda x.\, f\ (x\ x)).$$

It was discovered by Haskell Curry, and is one of the simplest fixed-point combinators. Note how similar its definition is to Ω.

Unfortunately, Y does not work under CBV. Evaluating $\mathsf{Y}\ G$ gives

$$\mathsf{Y}\ G \to (\lambda x.\, G\,(x\ x))\,(\lambda x.\, G\,(x\ x)) \to G\,((\lambda x.\, G\,(x\ x))\,(\lambda x.\, G\,(x\ x))) \to G\,(G\,((\lambda x.\, G\,(x\ x))\,(\lambda x.\, G\,(x\ x)))) \to \cdots$$

The argument to G is never a value, so evaluation never terminates. Instead, we'll use a slight variant of the Y combinator, called Z, which works under CBV. It wraps the self-application $x\ x$ in a λ, which delays it until it is needed. The Z combinator is defined as

$$\mathsf{Z} \triangleq \lambda f.\,(\lambda x.\,f\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,f\ (\lambda y.\,x\ x\ y))$$

Let's see it in action, on our function G. Define FACT to be Z G and calculate as follows:

$$\begin{array}{lll}
 & \mathsf{FACT} & \\
= & \mathsf{Z}\ G & \\
= & (\lambda f.\,(\lambda x.\,f\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,f\ (\lambda y.\,x\ x\ y)))\ G & \text{Definition of Z} \\
\to & (\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,G\ (\lambda y.\,x\ x\ y)) & \\
\to & G\ (\lambda y.\,(\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ y) & \\
= & (\lambda f.\,\lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (f\ (n-1))) & \\
 & \quad (\lambda y.\,(\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ y) & \text{definition of } G \\
\to & \lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1 & \\
 & \quad \textbf{else}\ n \times ((\lambda y.\,(\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,G\ (\lambda y.\,x\ x\ y))\ y)\ (n-1)) & \\
=_\beta & \lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\lambda y.\,(\mathsf{Z}\ G)\ y)\ (n-1) & \\
=_\beta & \lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\mathsf{Z}\ G\ (n-1)) & \\
= & \lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\mathsf{FACT}\ (n-1)) &
\end{array}$$

There are many (indeed infinitely many) fixed-point combinators. Here's a cute one, due to Jan Willem Klop:

$$\mathsf{Y}_k \triangleq (\mathsf{L}\ \mathsf{L})$$

where

$$\mathsf{L} \triangleq \lambda abcdefghijklmnopqstuvwxyzr.\,(r\ (t\ h\ i\ s\ i\ s\ a\ f\ i\ x\ e\ d\ p\ o\ i\ n\ t\ c\ o\ m\ b\ i\ n\ a\ t\ o\ r))$$

To gain some more intuition for fixed-point combinators, let's derive a fixed-point combinator that was originally discovered by Alan Turing [4]. Suppose we have a higher order function f, and want the fixed point of f. We want $\Theta\ f$ to be a fixed point of f, so we need

$$\Theta\ f = f\ (\Theta\ f).$$

This means, that we can write the following recursive equation:

$$\Theta = \lambda f.\,f\ (\Theta\ f).$$

Now we can use the recursion removal trick we described earlier. Define Θ' as $\lambda t.\,\lambda f.\,f\ (t\ t\ f)$, and Θ as $\Theta'\ \Theta'$. Then we have the following equalities:

$$\begin{array}{ll}
\Theta = \Theta'\ \Theta' & \\
\quad = (\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ \Theta' & \\
\quad \to \lambda f.\,f\ (\Theta'\ \Theta'\ f) & \\
\quad = \lambda f.\,f\ (\Theta\ f) &
\end{array}$$

Let's try out the Turing combinator on our higher order function G that we used to define FACT. This time we will use CBN evaluation.

$$
\begin{aligned}
\mathsf{FACT} &= \Theta\ G \\
&= ((\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ (\lambda t.\,\lambda f.\,f\ (t\ t\ f)))\ G \\
&\to (\lambda f.\,f\ ((\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ (\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ f))\ G \\
&\to G\ ((\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ (\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ G) \\
&= G\ (\Theta\ G) && \text{for brevity} \\
&= (\lambda f.\,\lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (f\ (n-1)))\ (\Theta\ G) && \text{Definition of } G \\
&\to \lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times ((\Theta\ G)\ (n-1)) \\
&= \lambda n.\,\textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\mathsf{FACT}\ (n-1))
\end{aligned}
$$

Note that the step from $G\ (\Theta\ G)$ passes the non-value $\Theta\ G$ to G, which is only allowed under CBN. Under CBV, $\Theta\ G$ diverges, just like $\mathsf{Y}\ G$. As with Z, we can fix this by delaying the self-application, using $\Theta'_v \triangleq \lambda t.\,\lambda f.\,f\ (\lambda y.\,t\ t\ f\ y)$ in place of Θ'.

References

[1] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, revised edition, 1984.

[2] A. Church and J. B. Rosser. Some properties of conversion. *Transactions of the American Mathematical Society*, 39(3):472–482, 1936.

[3] M. Takahashi. Parallel reductions in λ-calculus. *Information and Computation*, 118(1):120–127, 1995.

[4] A. M. Turing. The p-function in λ-K-conversion. *Journal of Symbolic Logic*, 2(4):164, 1937.